What's GILT?

GILT: Globalization, Internationalization, Localization, Translation

What is i18n testing?

●Internationalization (i18n) is the process of planning/designing/implementing a software application/websites/databases/products and services so that the underlying source code is localeneutral and it can easily be adapted to various languages and regions/cultures/locales without engineering changes, the result is one generic code that can be used across locales. It makes sure that the code can handle all international support without breaking functionality that would cause either data loss or display problems.

●i18n is a required step before actual product localization may begin.

●i18n testing is done to ensure that the software does not assume any specific language or conventions associated with a specific language and to ensure the software will work globally regardless of locale. It checks proper functionality of the product with any of the culture/locale settings using every type of international input possible.

i18n testing focus on?

- Localizability testing:

'a, use à or å c, use ĉ or ç n, use ń or ñ, aa+++some strings+++bb'

●UI message extract (hardcoding, concatenation, stripping, text fragmentation,ambiguity, fonts)

●UI layout (Alignment, truncation, overlap, dialog resizing, space limitations, image layering and size information)

●Over translation(ex. consistent, excessive technical jargon)

●Mirroring testing (ex. Righttoleft (RTL) text, mirrors windows/text alignment.)

●It does not test for localization quality.

- Input/output & functionality testing:

●Using Non-ASCII (non-English) characters (ex. East Asina languages, German, Complex Script characters: Arabic, Hebrew, Thai, and etc.)

●Install/Uninstall/Upgrade product on English/Localized OS

●Character sets/Encoding Support (Unicode&Native, ex.single byte character codes/set (SBCS) such as English, double byte/multiple byte character codes/set (DBCS/MBCS) such as Japanese Kanji, European characters such as German, bidirectional character set testing (BIDI) such as Hebrew or Arabic.)

●Multibyte Data input/output

●Functionality loss

- Locale testing:

●Formatting of Date/time, numbers, currency, calendars differences, measurement units, graphics, sorting, conversion, address, phone number, name, print, page size, Shortcut/hot key, bidi, white space, and etc.

●GUI/message display (ex. Garbage characters)

●Searching with non-ASCII characters

●IME (ex. Ibus & local IME such as kkc, libpinyin, and etc)

●Time Zone

●Display issues (ex. Question marks (?): Unicode-to-ANSI conversion; Random High ANSI characters (e.g., ¼, †, ‰, ‡, ¶): ANSI code using the wrong code page.; boxes, vertical bars, or tildes (default glyphs) [□, |, ~]: Selected font cannot display some of the characters.)

- Languages testing:

Focuses on testing the English product with a global environment of products and services functioning in non-English.

What is L10n testing ?

●Localization (L10n) is the process of adapting software for a specific region or language by adding locale -specific components and translating text.

●L10n testing is based on the results of globalization testing, which verifies the functional support for that particular culture/locale.

●L10n testing checks the quality of a product's localization for a particular target culture/locale. Localization testing can be executed only on the localized version of a product.

L10n testing focus on?

- Functionality testing:

●Basic functionality (locale-sensitive) tests on localized Env.

●Hot key, shortcut key, mnemonic key accessibility

●Check help via Help /F1 button

- Locale/Data format/Encoding testing:

●Consistency of formats: Date/time, numbers, currency, calendars differences, measurement units, graphics, sorting, conversion, address, phone number, name, print, page size, bidi, white space, and etc.

●DBCS/MBCS strings input/out, display

●Font changes, encoding/code-page issues

- UI/Cosmetic testing:

●Matching/Consistent with English version (printed documentation, online help, messages, interface resources, command-key sequences, etc.)

●Un-translated text

●Hard code

●Text expansion/ Text that takes up two lines

●Characters appear as empty squares

●Un-translated Tooltips

●Misalignment

●Truncation

●Overlap

●Layout/Appearance/Size

●String corruption/garble

●Punctuation

●Fields reversed or in the wrong order in right-to-left languages (such as Arabic, Urdu, Farsi, and Hebrew)

- Translation/Linguistic testing:

●All UI/Message strings are translated (manuals, online help, context help, etc.)

●Out-of-context translation errors

●Linguistic accuracy and resource attributes

●Inadvertently translated strings

●Typographical, grammatical, and contextual errors (e.g. Is the tab order from right to left in right to left languages?)

●Punctuation

●No over translated

●Terminology is used consistently in all documents and application UI